EUSKO JAURLARITZA
GOBIERNO VASCO

ELORRIETA ERREKA MARI CIFP LHII

2º DESARROLLO DE APLICACIONES WEB

Tema 2.11:
CSS Moderno

Diseño de Interfaces Web

CC BY NC SA

1. ¿Por qué CSS moderno?

- La web actual exige interfaces dinámicas, accesibles y responsivas.
- CSS ha evolucionado muchísimo entre 2020 y 2025.
- Nuevas herramientas reducen la necesidad de JavaScript.
- Maquetar hoy requiere dominar variables, color avanzado, unidades fluidas y nuevos selectores.

2. Custom Properties (Variables CSS)

¿Qué son las Custom Properties?

- Variables definidas directamente en CSS.
- Permiten reutilizar valores y mantener coherencia visual.
- Se pueden actualizar dinámicamente (incluyendo desde JavaScript).
- Funcionan en tiempo real y afectan al DOM sin recargar CSS.

3. Ventajas de las Variables CSS

Ventajas clave

- Facilitan temas claro/oscuro.
- Reducen duplicación de código.
- Se integran con calc(), clamp(), funciones de color, etc.
- Son heredables — encajan perfectamente con la cascada.

4. Ejemplo de Variables CSS

Idea clave: tu diseño depende ahora de una sola fuente de verdad.

```
:root {
  --color-primary: #2563eb;
  --padding: 1rem;
}

.card {
  padding: var(--padding);
  border-color: var(--color-primary);
}
```

5. Variables para tematización

Cambiar el tema con una sola clase

```
:root {
  --bg: white;
  --text: black;
}

.dark {
  --bg: #111;
  --text: #eee;
}

body {
  background: var(--bg);
  color: var(--text);
}
```

Perfecto para dark mode sin media queries.

6. Funciones modernas de color

CSS Color Level 4

- **Aporta nuevos espacios de color perceptuales.**
- **Incluye funciones matemáticas para mezclar, contrastar y generar colores.**
- **Mejora la accesibilidad desde el diseño, no desde plugins externos.**

7. Nuevos espacios de color

- **Espacios de color modernos:**
  - **lab()**
  - **lch()**
  - **oklab()**
  - **oklch() (muy utilizado en diseño actual)**
- **Ventajas**
  - **Transiciones más suaves.**
  - **Paletas más coherentes.**
  - **Mejor control de contraste.**

8. Funciones como color-mix()

color-mix() permite generar colores intermedios de forma precisa y declarativa.

- **Ejemplo:**

```
.button {
  background: color-mix(in srgb, #2563eb 70%, white);
}
```

Útil para hover, focus, variantes automáticas…

9. Unidades modernas

Unidades pensadas para resolver problemas reales

CSS moderno introduce unidades diseñadas para que el diseño se adapte de manera fluida a cualquier dispositivo.

Estas unidades permiten evitar media queries en muchos casos y solucionar problemas clásicos como el comportamiento inconsistente del viewport en móvil.

9. Unidades modernas

- **Antes**: px, %, rem, vw…
- **Ahora**: unidades diseñadas para resolver problemas reales.
  - dvh, svh, lvh: viewport real en móviles.
  - clamp() para tamaños fluidos con límites.
  - min(), max() para layouts adaptativos.
  - Combinación perfecta con media queries o sin ellas.

10. Problema del viewport en móviles

Durante años, el tamaño del viewport en móviles se veía afectado por barras de navegación, teclado, scroll…

Esto provocaba saltos visuales o secciones que no ocupaban el espacio esperado.

Solución (2023–2025):

Las nuevas unidades dvh, svh y lvh solucionan este problema ofreciendo valores claros y consistentes en función del estado del navegador.

- svh = altura segura
- dvh = altura dinámica real
- lvh = altura más amplia posible

11. Ejemplo de altura con dvh

Asignar **height: 100dvh** garantiza que ese elemento siempre ocupará el alto visible real, independientemente de que aparezcan barras o el teclado.

```
.hero {
  height: 100dvh;
}
```

Es la forma moderna y correcta de definir secciones a pantalla completa en dispositivos móviles.

12. Tipografía fluida con clamp()

La función **clamp()** permite crear tamaños de texto que se ajustan al ancho de pantalla sin necesidad de media queries.

Es una de las herramientas más usadas en diseño responsive actual, ya que facilita tipografías escalables y coherentes.

```
h1 {
  font-size: clamp(1.5rem, 5vw, 3rem);
}
```

Se puede aplicar no solo a fuentes, sino también a paddings, márgenes o tamaños de tarjetas.

13. Layouts adaptativos con min() y max()

min() y **max()** permiten definir tamaños que se ajustan dinámicamente según el espacio disponible.

```
.card {
  width: min(90%, 500px);
}
```

Esto permite controlar límites máximos y mínimos sin condicionales ni consultas de medios, creando componentes verdaderamente adaptativos.

14. Selectores modernos en CSS

Los selectores modernos permiten escribir CSS más limpio, semántico y potente.

Destacan tres pseudo-clases que cambian por completo la forma de abordar el estilo de componentes: **:is()**, **:where()** y el revolucionario **:has()**.

Estas herramientas reducen código duplicado y permiten estilos basados en estructura y estado sin usar JavaScript.

15. :is() y :where() para simplificar selectores

:is() y **:where()** permiten agrupar múltiples selectores en una única expresión, optimizando la legibilidad y evitando repeticiones.

La diferencia clave es la **especificidad**: **:is()** mantiene la especificidad de lo que envuelve, mientras que **:where()** siempre tiene especificidad cero, ideal para resets o estilos generales.

```
:is(header, footer, main) p {
  margin: 0;
}
```

16. :has(): el selector más avanzado

:has() permite seleccionar elementos padres en función de lo que contienen.

Esto abre la puerta a patrones tradicionales de UI sin necesidad de JavaScript: resaltar una caja si contiene ciertos elementos, modificar un contenedor según el estado de un input, etc.

```
.card:has(img) {
  padding-top: 0;
}
```

Es uno de los avances más importantes de CSS en la última década.

17. :has() como alternativa real a JavaScript

Antes, muchas dinámicas dependían de consultar el DOM mediante JavaScript.

Hoy, CSS puede detectar si un elemento existe, si está en cierto estado, o incluso si un enlace está activo dentro de un menú.

Esto reduce el peso del proyecto, simplifica la arquitectura y mejora el rendimiento.

FIN